

Lab 11 — Trees and the Four Walks

DSA27 Lab Manual · Week 11 · Data Structures and Algorithms

Dr. Haitham A. El-Ghareeb

Fall 2026 · Week 11

Contents

Part 0 — Before you start	2
Part 1 — insert and contains: one path down	4
Part 2 — See your tree; min and max	6
Part 3 — height and size: recursion on trees	7
Part 4 — is_valid: the whole subtree, not just the parent	8
Part 5 — Three walks, one line apart	9
Part 6 — level_order: a queue, not a call stack	10
Part 7 — delete: three cases	12
Part 8 — Measure it	13
Part 9 — Exercises at a glance	15
Part 10 — Take-home practice (not graded)	15
Part 11 — Bridge to Lecture 12: the tree in an array	16
Summary	16
Answers to the checkpoints	17

How to use this lab. The routine from Week 4: read the lecture section named at the top of each part, **draw before you code** — this week that means drawing the tree, with every link, on paper — and predict at each **Checkpoint** (answers at the end). Then write one method, run the tests named in its part, and only then move on. One tree is used all the way through, so the drawings you make in Part 1 are the ones you check everything else against. Parts 1–7 are the session; Parts 8–11 are for home.

Duration	One 2-hour lab session, plus about 3 hours at home
----------	--

You will write**Graded by**
Connects to

dsa/tree.py — BinarySearchTree:
insert, contains, min, max, height,
size, is_valid, in_order, pre_order,
post_order, level_order, delete
tests/test_tree.py (26 tests)
Lecture 11 — Trees; Lab 07's
CircularQueue, which level_order uses;
and Lecture 12, where a tree lives in an
array

What you will be able to do

1. draw the BST that a sequence of inserts builds, and say which insertion order builds a chain;
 2. write insert and contains as loops that walk one path down, and explain why insert must stop **at the parent**;
 3. write recursive height and size from a base case and a recursive case, and say why the empty tree has height -1 ;
 4. explain why checking each parent against its children does not prove a tree is a BST, and write is_valid with a range;
 5. write the three recursive traversals, and say what each order is **for**;
 6. write level_order with your own CircularQueue, and trace the queue;
 7. write delete for all three cases, including the root, and draw the successor case step by step;
 8. measure the height of a BST built from sorted and from shuffled input, and meet the recursion limit.
-

Part 0 — Before you start**0.1 Environment and prerequisite**

From the DSA27 folder, with the virtual environment active (the prompt starts with (.venv)):

```
git pull
pytest -m "not challenge" -q          # the environment check: must pass
pytest tests/test_stack_queue.py -q   # Weeks 6-7: 18 passed
pytest tests/test_tree.py -q          # 26 failures: nothing yet
```

level_order, in Part 6, imports **your** CircularQueue from dsa/queue.py. It must **grow** when it is full — the challenge of Lab 07, Part 6 — because test_level_order_on_a_wide_tree has 16 nodes waiting at once and the default capacity is 8. The provided queue tests never overfill a ring, so check the growth yourself now:

```
>>> from dsa.queue import CircularQueue
>>> q = CircularQueue(2)
>>> for v in range(5):
...     q.enqueue(v)
...
>>> q.dequeue(), list(q)
(0, [1, 2, 3, 4])
```

If that raises “queue is full”, finish Lab 07’s Part 6 before Part 6 of this lab. Everything before Part 6 works without it.

0.2 Read the skeleton: what is given, what is yours

Open `dsa/tree.py` and read all of it — the module docstring first, which warns you about the linked list “wearing a tree costume”.

Given to you	Yours
TreeNode — value, left, right	insert, contains, delete
to_edges(node) — the (parent, child) pairs for draw_tree	min, max, height, size, is_valid
BinarySearchTree.__init__(values=())	in_order, pre_order, post_order, level_order
— sets self.root = None, then calls your insert on each value	
__len__, __contains__, __iter__, __repr__ — they call your methods	

The tree stores exactly one thing: `self.root`, a `TreeNode` or `None`. Every method starts from there.

Because `__init__` calls your `insert`, **every** test fails until `insert` works. That is why it is Part 1.

0.3 The storage rule, this week

A `dsa/` structure keeps its data only in the course `Array`, in node objects, or in another `dsa/` structure (Lecture 02). A BST keeps its data in `TreeNodes`, and nowhere else:

- no `self._values = []` beside the tree, and no sorted list that `in_order` simply returns;
- the four traversals **return** a new Python list each time. That list is **output**, made for the caller — the tree does not keep it (`test_traversals_return_a_fresh_list`);
- `level_order` needs a queue as working storage: your `CircularQueue`, not a Python list with `pop(0)` and not `collections.deque`.

0.4 The tests, and one warning about -k

-k matches the **file** name as well as the test name, and the file is `test_tree.py`. So `-k tree` selects all 26 tests. Filter by the method instead:

Filter	Selects
-k "insert or contains or dup or struct"	the 5 building tests
-k "min"	the 2 min/max tests
-k "height or empty or degen"	the size-and-height test, the degenerate-tree test, and the three tests with "empty" in their names (5)
-k valid	the broken-tree test for <code>is_valid</code> (1)
-k "order or traversals"	the 8 traversal tests
-k delete	the 6 delete tests

The tree used by most tests is drawn in a comment at the top of `tests/test_tree.py`. It is the lecture's tree, built from `[8, 3, 10, 1, 6, 14, 4, 7, 13]`. This lab uses a **different** tree, so that your drawings are your own work.

Part 1 — insert and contains: one path down

Lecture 11, "Search: one path down" and "Insert: a search that fails".

1.1 The idea

At each node, compare: smaller goes **left**, larger goes **right**, equal is found. A search walks one path from the root until it finds the value or steps onto `None`. An insert is the same walk; where the search would have stepped onto `None`, the new node is hung instead. Nothing moves, and the new node is always a leaf.

1.2 Draw it

Take a sheet of paper. This is **the lab tree**, used in every part:

```
T = [15, 6, 20, 3, 9, 18, 24, 7, 12, 22]
```

Insert the values one at a time, left to right, into an empty tree. For each, write the path of comparisons — "9: < 15 left, > 6 right, None: here" — and draw the node where it lands. Draw the links as arrows from parent to child, and leave room: you will add to this drawing all afternoon.

Checkpoint 1. (a) Draw *T*. What is its height? Which nodes are leaves? (b) Now insert 8 and then 19 into your drawing. For each, list the nodes compared on the way down, and

say whose left or right changes. (c) Draw `BinarySearchTree([1, 2, 3])` and `BinarySearchTree([2, 1, 3])`. Same values — what is different, and why?

1.3 Write it

contains(value) — a loop, not a recursion:

1. Start with a variable `node` at the root.
2. While `node` is not `None`: equal $\rightarrow$ `True`; smaller $\rightarrow$ move to `node.left`; larger $\rightarrow$ move to `node.right`.
3. The loop ended on `None`: `False`.

insert(value):

1. Empty tree: the new node becomes `self.root`. Done.
2. Otherwise walk down from the root as in `contains`. If you meet the value, return — duplicates are ignored.
3. Before stepping to a child, **look at it**: if the child you are about to take is `None`, set that link — `node.left` or `node.right` — to the new node, and return.

Step 3 is the whole trick. Walk until `node` is `None` and it is too late: you are holding `None`, not the parent, and `node = TreeNode(value)` changes a local variable that nothing else can see.

1.4 Test it

```
pytest tests/test_tree.py -v -k "insert or contains or dup or struct"
```

Five tests: `test_insert_and_contains` (every value of the sample tree is found; 0, 2, 5, 9 and 15 are not), `test_structure_is_actually_a_bst`, which checks the **links** of [8, 3, 10], not just the values, and `test_to_edges_matches_the_structure`, which reads the same links through the given `to_edges` — these three must pass now. The other two also call methods you have not written yet, so they fail with `NotImplementedError` until then: `test_duplicates_are_ignored` needs `size` (Part 3) and `in_order` (Part 5), and `test_insert_keeps_the_invariant` needs `is_valid` (Part 4). Expect **3 passed, 2 failed** — that is right for now.

1.5 When it fails

Bug	What you see	Fix
walking until node is None, then node = TreeNode(value)	nearly everything: assert 1 == 2 in the duplicates test, AttributeError: 'NoneType' object has no attribute 'value' in the structure test — the tree never grows past its root	stop at the parent: test the child before you step onto it
no equality check: equal values go right	assert 5 == 2 in the duplicates test (once size works)	value == node.value → return
forgetting the empty tree	AttributeError: 'NoneType' object has no attribute 'value' on the first insert	if self.root is None, the new node is the root
contains returns inside the loop on the first comparison	assert False in the insert-and-contains test, for a value deeper down	return only when equal; otherwise move and keep going

Part 2 — See your tree; min and max

Lecture 11, “Min and max: no comparisons at all”.

2.1 Draw it — by machine, this time

With insert working, let the course tools draw what your code built. In notebooks/11-trees.ipynb:

```
from viz.draw import draw_tree
from dsa.tree import BinarySearchTree, to_edges

T = [15, 6, 20, 3, 9, 18, 24, 7, 12, 22]
bst = BinarySearchTree(T)
draw_tree(to_edges(bst.root), highlight={"15"})
```

Compare it with your paper drawing, link by link. If they differ, one of them is wrong — find out which before you go on. (to_edges is given, and uses only value, left and right: it shows your links exactly as they are.)

A limitation worth knowing: Graphviz lays out a node with **one** child directly below it, so it does not show whether that child is a left or a right child. For that, read the edges: to_edges(bst.root) lists each parent’s left child before its right child.

2.2 Write it

min(): if the tree is empty, raise `ValueError`. Otherwise start at the root and go **left** while there is a left child; return the value where you stop. **max()** is the mirror image.

Checkpoint 2. For the lab tree *T*: which nodes does `min()` visit, and which does `max()`? How many times does either compare two values?

```
pytest tests/test_tree.py -v -k min
```

Two tests. The one that fails most often is `test_min_max_on_empty_raise`: returning `None` for an empty tree gives `Failed: DID NOT RAISE ValueError`. An empty tree has no minimum; `None` would be a lie that the caller might store and use.

Part 3 — height and size: recursion on trees

Lecture 11, “Depth and height” and “Height and size: post-order in one line”.

3.1 The idea

A tree is a root plus two smaller trees. So a question about a tree can often be answered from the answers for its two subtrees:

- **size** of a tree = 1 + size of the left subtree + size of the right;
- **height** of a tree = 1 + the larger of the two subtrees’ heights;
- and the empty tree answers directly: size 0, height -1 .

Write each as a public method that calls a private recursive helper on `self.root` — `height()` calls `self._height(self.root)`. The helper takes a **node**, because the recursion is on subtrees, and a subtree is a node.

3.2 Checkpoint

Checkpoint 3. Give `height()` and `size()` for the trees built from: (a) `[]`; (b) `[5]`; (c) `[1, 2, 3, 4, 5]`; (d) `[3, 1, 5, 2, 4]`; (e) `[4, 2, 6, 1, 3, 5, 7]`; (f) `[5, 4, 3, 2, 1, 6]`; (g) the lab tree *T*. Then: if `_height(None)` returned 0 instead of -1 , what would each height become?

3.3 Write it and test it

```
pytest tests/test_tree.py -v -k "height or empty or degen"
```

Five tests. Expect **3 passed, 2 failed**: `test_empty_tree` also checks `in_order`, and `test_traversals_of_an_empty_tree` checks all four walks, so both wait for Parts 5 and 6. (`test_min_max_on_empty_raise` is the third “empty” test; it passed in Part 2.)

Bug	What you see	Fix
<code>_height(None)</code> returns 0	<code>assert 0 == -1</code> in the empty-tree test, <code>assert 4 == 3</code> in the size-and-height test, <code>assert 5 == 4</code> in the degenerate test — every height one too big	the empty tree is <code>-1</code> : height counts edges
<code>1 + max(...)</code> in <code>_size</code>	<code>assert 4 == 9</code> in the size-and-height test: that is the number of nodes on the longest path , not the number of nodes	add both subtrees: <code>1 + left + right</code>
the helper recurses on <code>self.root</code> instead of the node it was given	<code>RecursionError: maximum recursion depth exceeded</code>	pass <code>node.left</code> and <code>node.right</code>

Part 4 — `is_valid`: the whole subtree, not just the parent

Lecture 11, “The BST property” and “`is_valid`: carry the range down”.

4.1 The idea

The tempting check — at each node, `left.value < node.value < right.value` — looks only one level down. The BST property talks about **whole subtrees**. A value deep in the left subtree of 15 must be smaller than 15, even if its own parent is 6.

So pass a range (**low, high**) down the recursion. At the root, there is no limit on either side (use `None`). Going left from a node, the node’s value becomes the new high; going right, the new low. A node whose value is not strictly inside its range breaks the tree.

4.2 Draw it

On your drawing of T, write beside each node the range its ancestors allow, as the lecture’s figure does: 15 gets $(-\infty, \infty)$, 6 gets $(-\infty, 15)$, 9 gets $(6, 15)$, 12 gets $(9, 15)$, and so on.

Checkpoint 4. Four hand-built trees. For each, give (i) what `is_valid()` returns and (ii) what the parent-only check returns:

```
A = TreeNode(10, TreeNode(5, None, TreeNode(12)), TreeNode(15))
B = TreeNode(10, TreeNode(5, TreeNode(2), TreeNode(7)),
              TreeNode(15, TreeNode(11), TreeNode(20)))
C = TreeNode(10, TreeNode(5, None, TreeNode(10)), None)
D = TreeNode(10, TreeNode(5, TreeNode(2), TreeNode(7)),
              TreeNode(15, TreeNode(9), TreeNode(20)))
```

4.3 Write it and test it

A helper `_is_valid(node, low, high)`: True for None; False if the value is at or below low (when there is a low) or at or above high (when there is a high); otherwise both subtrees, with the narrowed ranges.

```
pytest tests/test_tree.py -v -k "valid or invariant"
```

Two tests. `test_is_valid_catches_a_hand_built_broken_tree` builds the lecture's broken tree (9 in the left subtree of 8). The parent-only check fails it with `assert not True` — it reports the broken tree as valid.

Two quieter bugs the test cannot see: writing `if low and node.value <= low` treats a bound of **0** as “no bound”, since `0` is falsy (write `low is not None`); and using `<` where `<=` belongs lets a duplicate of an ancestor through, as in tree C.

Part 5 — Three walks, one line apart

Lecture 11, “Recursion on trees”, “In-order: the code” and “What each one is FOR”.

5.1 The idea

The three recursive traversals have the same skeleton: do nothing for None; otherwise handle the node and walk both subtrees, left before right. Where you put “handle the node” — **before**, **between** or **after** the two walks — gives pre-order, in-order and post-order.

Each public method makes one output list, calls a helper with the root and that list, and returns the list. The helper appends to the **same** list at every level; it does not build and return lists of its own.

5.2 Draw it

Checkpoint 5. Write down, by hand, the four walks of the lab tree *T*: pre-order, in-order, post-order and level-order. Then answer: (a) Which one is sorted, and why must it be? (b) Insert the **pre-order** list into a new, empty tree. Is the result the same shape as *T*? What if you insert the **in-order** list instead? (c) Which node is always first in pre-order, and always last in post-order?

5.3 Write it

Write `in_order` with its helper `_in_order(self, node, out)` exactly as in the lecture, run its test, then write `pre_order` and `post_order` by moving one line.

```
pytest tests/test_tree.py -v -k "order or traversals"
```

Eight tests. Expect **4 passed, 4 failed**: the two level-order tests, and the two that try all four walks (`test_traversals_return_a_fresh_list`, `test_traversals_of_an_empty_tree`), wait for Part 6. The pre-order test rebuilds the tree from its own pre-order list and compares the walks.

5.4 When it fails

Bug	What you see	Fix
the helper starts with its own <code>out = []</code>	<code>assert [] == [1, 3, 4, 6, ...]</code> — every value goes into a list that is thrown away, and the caller's list stays empty	make the list once, in the public method, and pass it down
the list kept on the tree (<code>self._out</code>) and returned every time	<code>AssertionError: in_order handed out a list the tree still uses</code>	a new list per call: a traversal is output, not storage
<code>left_list + [node.value] + right_list</code>	the tests pass	correct, but every level copies its whole subtree: $O(n^2)$ on a chain. Use one list
visiting right before left	<code>assert [8, 10, 14, 13, 3, 6, ...] == [8, 3, 1, 6, 4, 7, ...]</code> in the pre-order test	left always before right, in all three orders

Part 6 — level_order: a queue, not a call stack

Lecture 11, "Level-order needs a queue".

6.1 The idea

Level-order visits nodes in order of their distance from the root. The call stack cannot do that — it always goes deeper into the most recent node, last in, first out. A **queue** keeps the discovered nodes in the order they were found, first in, first out:

1. empty tree → return an empty list;

2. enqueue the root;
3. while the queue is not empty: dequeue a node, append its value, enqueue its left child and then its right child (only those that are not None).

The queue holds **nodes**, not values: you need a node's children later, and a value does not have any.

6.2 Draw it

Draw the queue as a row of boxes, front on the left, after every dequeue — the format of the lecture's figure.

Checkpoint 6. (a) Trace `level_order` on the lab tree *T*: after each dequeue, which values are in the queue? What is the largest number of nodes in the queue at any one time? (b) The test builds a perfect tree of 31 nodes. How many nodes are in the queue at its fullest? Why does that break a `CircularQueue` that cannot grow?

6.3 Write it and test it

The skeleton already imports it, at the top of `dsa/tree.py`: `from dsa.queue import CircularQueue`. Make one with `CircularQueue()` and let it grow.

```
pytest tests/test_tree.py -v -k level
```

Two tests: `test_level_order` on the sample tree, and `test_level_order_on_a_wide_tree`.

Bug	What you see	Fix
a queue that cannot grow	<code>IndexError: queue is full</code> in the wide-tree test only	finish Lab 07's growth challenge; do not just pass a big capacity to hide it
right child enqueued before the left	<code>assert [8, 10, 3, 14, 6, 1, ...] == [8, 3, 10, 1, 6, 14, ...]</code> , and <code>[16, 24, 8] == [16, 8, 24]</code>	left first
a Python list with <code>pop(0)</code>	the tests pass	the storage rule, and $O(n)$ per dequeue — Lecture 07's <code>SlowQueue</code> . The TA will read your code
no empty-tree check	<code>AttributeError: 'NoneType' object has no attribute 'value'</code> in the empty-traversals test — <code>None</code> was enqueued as if it were the root	<code>return []</code> before enqueueing anything

Try this. Replace the queue by your Stack from Week 6 — push instead of enqueue, pop instead of dequeue — and push the **right** child before the left. Which of the three recursive orders do you get? (Answer in Part 6 of the checkpoint answers — and Week 14 makes this the difference between BFS and DFS.)

Part 7 — delete: three cases

Lecture 11, “Delete: Three Cases”. This is the method of the week. Draw every step on paper before you write a line, and redraw the tree after every delete.

7.1 The idea

1. **Find** the node, and keep a variable for its **parent** as you walk. Not found: return.
2. **Two children?** Find the in-order successor: go right once, then left until there is no left child — keeping *its* parent too. Copy the successor’s value into the node. From now on, the node to remove is the **successor**, with its parent.
3. The node to remove now has **at most one** child. Call it *child* (it may be None). Put *child* where the node was: in `self.root` if the node has no parent; otherwise in whichever of the parent’s links — `left` or `right` — points at the node.

Step 2 always hands step 3 an easy case: the successor was found by going left until there was no left child, so it has no left child.

7.2 Draw it

Start again from a fresh drawing of *T*.

Checkpoint 7. Delete, in this order, redrawing the tree after each: **3**, then **6**, then **15**, then **20**. For each delete, name the case (*leaf*, *one child*, *two children*), and for the two-children case name the successor and the path to it. Give the height of the tree at the end.

Checkpoint 8. In the **original** *T*, what would `delete(9)` do? Which node is the successor, how many times does the “go left” loop run, and which link changes in step 3 — a `left` or a `right`?

7.3 Write it

Follow 7.1. Two hints:

- To know which link of the parent to change, ask `parent.left is node` — an identity test, “does this link point at this node?”.
- After step 2, **reassign** your two variables, the node and its parent, to the successor and the successor’s parent. Then step 3 is written once and serves all three cases.

7.4 Test it

```
pytest tests/test_tree.py -v -k delete
```

Six tests: a leaf (1), one child (14), two children (3), the root (8), a missing value (999), and every value of the sample tree deleted in turn, with `is_valid()` checked after each one.

7.5 When it fails

Bug	What you see	Fix
no root case: always <code>parent.left = ...</code> or <code>parent.right = ...</code>	<code>AttributeError:</code> 'NoneType' object has no attribute 'left' in the delete-everything test only	when the parent is <code>None</code> , the node is the root: <code>self.root = child</code>
two children: <code>successor_parent.left</code> <code>= successor.right</code> , always	the root test and the delete-everything test fail at <code>assert bst.is_valid()</code> ; the two-children test passes	when the successor is the node's own right child, the link to change is the node's right. Reassigning to the successor and using step 3 handles both
the value is copied, but the successor is never removed	the two-children test, the root test and delete-everything fail at <code>assert bst.is_valid()</code> — the value is now in the tree twice	step 3 must run on the successor
a missing value crashes	<code>AttributeError:</code> 'NoneType' object has no attribute 'left' in the missing-value test	if the search ends on <code>None</code> , return

The first bug is worth a minute. Why does `test_delete_the_root` **pass** with it? Because in the sample tree the root, 8, has two children: its parent is never needed, since step 2 moves the removal down to the successor. The root case only appears when the root has **at most one** child — which happens late in `test_delete_everything`, when the tree has shrunk.

Part 8 — Measure it

Lecture 11, “When the Tree Is a Linked List” and “Measured”.

8.1 Height against insertion order

```
import random
from dsa.tree import BinarySearchTree

rng = random.Random(1)
for n in [100, 200, 400, 800]:
    ordered = list(range(n))
    shuffled = ordered[:]
    rng.shuffle(shuffled)
    print(n, BinarySearchTree(ordered).height(),
          BinarySearchTree(shuffled).height())
```

Checkpoint 9. Predict the first height column exactly, and the second roughly. Then run it. What would `BinarySearchTree(range(1000)).height()` do?

8.2 Time a search

```
from viz.complexity import measure, plot_growth

sizes = [100, 200, 400, 800]
def build(n, shuffled):
    values = list(range(n))
    if shuffled:
        random.Random(n).shuffle(values)
    return BinarySearchTree(values), n - 1    # look for the largest

search = lambda a: a[0].contains(a[1])
ordered = measure(search, sizes, lambda n: build(n, False))
random_ = measure(search, sizes, lambda n: build(n, True))
plot_growth({'sorted input': ordered, 'shuffled input': random_},
            reference=['n', 'log n'], loglog=True)
```

`measure` builds each tree outside the timing, so only the search is timed. On sorted input, searching for the largest value walks every node: a line of slope 1. On shuffled input it walks one short path, and the line is almost flat — and so small that timer noise shows. The lecture’s figure averages 2,000 searches per point to smooth that out.

8.3 The recursion limit

Now build `BinarySearchTree(range(1000))` and call `height()`. Read the error, then call `contains(999)` and `level_order()` on the same tree. Two of the three work. Why exactly those two? (Lecture 11, “Measured”).

Part 9 — Exercises at a glance

Method	Target cost	The trap	Tests
insert	$O(h)$	stop at the parent ; duplicates ignored; empty tree	-k "insert or contains or dup or struct"
contains	$O(h)$	a loop; move on, do not return early	as above
min, max	$O(h)$	ValueError when empty	-k min
height, size	$O(n)$	empty tree -1 and 0	-k "height or empty or degen"
is_valid	$O(n)$	carry (low, high); is not None, not truthiness	-k valid
three recursive walks	$O(n)$	one list, passed down; a new list per call	-k "order or traversals"
level_order	$O(n)$	your growing CircularQueue; left before right	-k level
delete	$O(h)$	the parent; the root; successor = the right child	-k delete

All 26 at once:

```
pytest tests/test_tree.py -v
```

Before you show the TA: `git diff --stat tests/` must print nothing. And check by eye that nothing in `dsa/tree.py` keeps a Python list, set or dict as an attribute, and that `level_order` uses `CircularQueue`.

Part 10 — Take-home practice (not graded)

The question bank for this week is `docs/question-bank/week11-questions.md`, with answers in `week11-answers.md`. Do the questions before opening the answers.

1. **Part G — write the code**, in `practice/week11.py`: `count_leaves`, `is_balanced`, `range_values`, `build_balanced` and `lowest_common_ancestor` (W11-C1 to W11-C5). They take a `TreeNode` root, not a `BinarySearchTree`, and the tests count how many nodes you read — so an answer that walks the whole tree when it needs one path fails:

```
pytest tests/test_practice_week11.py -v
```

Hint for `range_values`: it is your `_in_order`, with two ifs that skip a subtree the BST property says cannot hold an answer.

2. **W11-S1** — draw a BST after each of ten inserts, then after four deletes that cover all three cases.
3. **W11-T3** — rebuild a tree from its pre-order and in-order walks.
4. **W11-B3** — a delete that passes the two-children test and still breaks the tree. Find the input.
5. **W11-K2** — a height that calls itself twice on the taller child. It is correct; count its calls on a chain of 20 nodes.

The worked solutions are in `solutions/dsa/tree.py` and `solutions/practice/week11.py` — for after you have tried. `pytest --solutions tests/test_practice_week11.py` runs the tests on them.

Part 11 — Bridge to Lecture 12: the tree in an array

This week's tree is made of nodes and links, and its shape depends on the order the values arrive in. Next week's is made of **no** links at all.

Number the nodes of a **complete** binary tree level by level, left to right, starting from 0: the root is 0, its children 1 and 2, theirs 3, 4, 5, 6. Then the children of node i are always $2i + 1$ and $2i + 2$, and its parent is $(i - 1) // 2$. So the tree can live in a plain Array, in level order — which is exactly the order your `level_order` produces.

```
from viz.draw import draw_array_as_tree
draw_array_as_tree([1, 3, 2, 7, 4, 5, 9], highlight=[0])
```

Draw it, and check two things by hand: node 1 (value 3) has children at 3 and 4 (values 7 and 4); node 5 (value 5) has its parent at $(5 - 1) // 2 = 2$ (value 2). Every parent in that picture is smaller than its children. That is the **heap property** — weaker than the BST property, and in exchange the shape is always complete, so the height is always $\lfloor \log_2 n \rfloor$, whatever order the values came in. Lecture 12 builds a priority queue on it in `dsa/heap.py`.

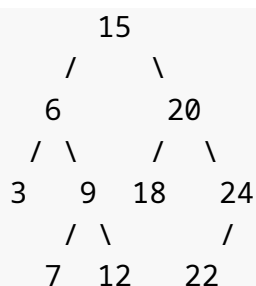
Summary

Idea	The one line to keep
The tree	One root; each other node has one parent; <code>self.root</code> is all the tree stores.

Idea	The one line to keep
Height	Edges on the longest root-to-leaf path; empty tree — 1, leaf 0.
BST property	Left subtree smaller, right subtree larger — check it with a range.
Search, insert	One path down, $O(h)$; insert stops at the parent and hangs a new leaf.
Min, max	Left (right) until you cannot; no comparisons.
Recursion	Base case None; answer from the two subtrees' answers.
Traversals	In-order sorted; pre-order rebuilds; post-order frees and evaluates.
Level-order	A queue of nodes — your growing CircularQueue.
Delete	Leaf: cut. One child: splice. Two: copy the successor, delete it instead.
The root	The one node with no parent: <code>self.root = child</code> .
Measured	Sorted input: height $n - 1$, $O(n)$ search, and a RecursionError at 1,000.

Answers to the checkpoints

Checkpoint 1. (a) The lab tree T:



Height 3, along $15 \rightarrow 6 \rightarrow 9 \rightarrow 7$ (and two other paths of the same length, $15 \rightarrow 6 \rightarrow 9 \rightarrow 12$ and $15 \rightarrow 20 \rightarrow 24 \rightarrow 22$). Leaves: 3, 7, 12, 18 and 22. (b) **8**: compared with 15 (left), 6 (right), 9 (left), 7 (right) — 7 has no right child, so `7.right` becomes the new node. **19**: compared with 15 (right), 20 (left), 18 (right) — `18.right` becomes the new node. The height is now 4, along $15 \rightarrow 6 \rightarrow 9 \rightarrow 7 \rightarrow 8$. (c) `[1, 2, 3]` builds a chain: 1 at the root, 2 its right child, 3 the right child of 2 — height 2. `[2, 1, 3]` puts 2 at the root with 1 and 3 as its two children — height 1. The first value always becomes the root, and each value goes where the values **before** it send it.

Checkpoint 2. `min()` visits 15, 6, 3 and returns 3; `max()` visits 15, 20, 24 and returns 24.

Neither compares two values even once: each only asks “is there a left (right) child?”. The comparisons were all done by insert.

Checkpoint 3.

Tree	height	size
(a) []	−1	0
(b) [5]	0	1
(c) [1, 2, 3, 4, 5]	4	5
(d) [3, 1, 5, 2, 4]	2	5
(e) [4, 2, 6, 1, 3, 5, 7]	2	7
(f) [5, 4, 3, 2, 1, 6]	4	6
(g) T	3	10

With `_height(None) == 0`, every tree except the empty one would come out one higher — (b) would be 1, (c) 5, and so on — because each leaf would count itself. The empty tree would be 0, the same as a single node: the convention could no longer tell them apart.

Checkpoint 4.

Tree	is_valid()	parent-only check	Why
A	False	True	12 is in 10's left subtree
B	True	True	every range holds: 7 in (5, 10), 11 in (10, 15)
C	False	True	the second 10 is in the left subtree of the first; the range is (5, 10), open at 10
D	False	True	9 is in 10's right subtree; its range is (10, 15)

The parent-only check passes all four. Only B is a BST: its in-order walk, 2, 5, 7, 10, 11, 15, 20, is the only one of the four that is strictly increasing.

Checkpoint 5.

pre-order	15, 6, 3, 9, 7, 12, 20, 18, 24, 22
in-order	3, 6, 7, 9, 12, 15, 18, 20, 22, 24
post-order	3, 7, 12, 9, 6, 18, 22, 24, 20, 15
level-order	15, 6, 20, 3, 9, 18, 24, 7, 12, 22

- (a) In-order is sorted. For every node, the values of its left subtree are smaller and are all written before it; those of its right subtree are larger and are all written after it.

- (b) Re-inserting the pre-order list rebuilds exactly T: each value arrives after all its ancestors, so it follows the same path and lands in the same place. Re-inserting the in-order list builds a **chain** of height 9: it is sorted, so each value is the largest so far and goes right every time.
- (c) The root, 15: first in pre-order, last in post-order. (Here level-order happens to equal the insertion order of T, because T was listed level by level.)

Checkpoint 6. (a)

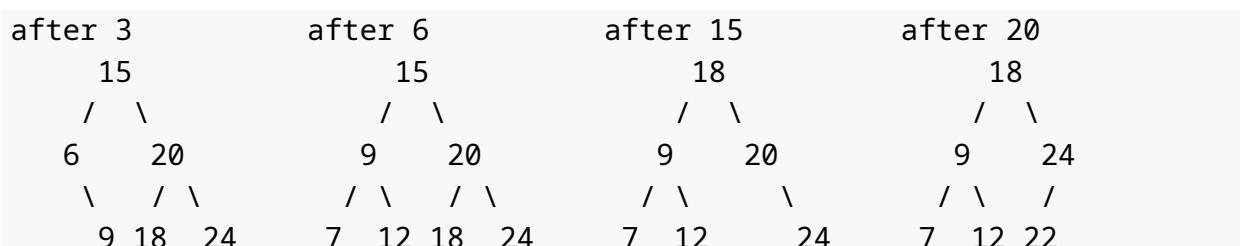
Dequeued	Queue after the step
(start)	15
15	6, 20
6	20, 3, 9
20	3, 9, 18, 24
3	9, 18, 24
9	18, 24, 7, 12
18	24, 7, 12
24	7, 12, 22
7	12, 22
12	22
22	(empty)

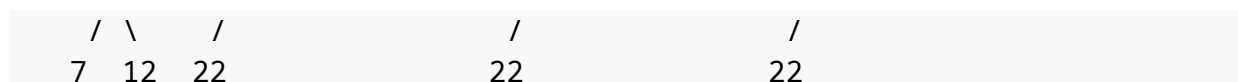
At most **4** nodes wait at once. (b) **16**: when the last node of level 3 has been dequeued, all 16 leaves are waiting. A `CircularQueue` of the default capacity 8 that cannot grow raises on the ninth enqueue. In general a complete tree's last level holds about half of its nodes, so the queue must be able to hold about $n/2$.

Try this (Part 6). A stack that pushes the right child before the left pops the left child first, and so visits node, left subtree, right subtree: **pre-order**, 15, 6, 3, 9, 7, 12, 20, 18, 24, 22 for T.

Checkpoint 7.

- **delete 3** — a **leaf**: `6.left = None`.
- **delete 6** — now **one child** (9): `15.left = 9`. The whole subtree 9, 7, 12 moves up one level.
- **delete 15** — the root, **two children**. Successor: right once to 20, then left to 18; 18 has no left child, so it is the successor (and it is a leaf). Copy 18 into the root, then remove the old 18: `20.left = None`.
- **delete 20** — now **one child** (24): `18.right = 24`.





Every step leaves a valid BST. Final height: **2**.

Checkpoint 8. 9 has two children, 7 and 12. The successor: right once to 12; 12 has no left child, so the “go left” loop runs **zero** times and 12 — 9’s own right child — is the successor. Its value is copied into 9’s node; then the old 12 is removed with the one-child rule, and since it hangs from the node that held 9 by that node’s **right** link, the link that changes is a **right**: the node now holding 12 gets `right = None` (12 had no children). The result has 12 where 9 was, with 7 as its left child. This is the case the “always `successor_parent.left`” bug gets wrong.

Checkpoint 9. The sorted column is exactly $n - 1$: 99, 199, 399, 799. The shuffled column grows like a logarithm; with the reference solution and `random.Random(1)` it prints 11, 13, 18, 18 — a random tree’s height is itself random, so it need not rise every time, but it stays within a small multiple of $\log_2 n$ (about 7 to 10 here). `BinarySearchTree(range(1000)).height()` raises `RecursionError: maximum recursion depth exceeded: the recursion is 1,000 calls deep, one per level, and Python’s default limit is 1,000 frames`.